

Mandate, the build plan

Read this top to bottom once, then work from the checklist. Everything here is settled, `CONTEXT.md` , `THE-PLAN.md` and `THE-IDEA.md` now agree with each other.

The pitch, one sentence

An agent gets a scoped virtual Rain card bound to the exact purchase order it negotiated, vendor, SKU, price, expiry. Any deviation means no card is ever issued.

If a judge compresses the six checks to “a foreign key and a uniqueness constraint before the API call”: **this is a pre-issuance three-way match**, the same PO/receipt/invoice reconciliation every ERP does, moved from after the invoice arrives to before the card exists at all.

Do these two, right now, before writing another line of checker code

- 1. **Storage.** Pick Neon Postgres or Vercel KV/Upstash. In-memory or on-disk state does not survive a Vercel cold start, and the append-only decision log needs to survive one, or replay silently breaks on the exact URL you submit. B decides, five minutes, move on.
- 2. **Ask a Rain engineer, together, before 14:00:**
 - Can we simulate an authorization against a card we issue?
 - Does card issuance lock to one exact merchant, or only a category?
 - Base URL and auth header for this weekend.
 - Is the collateral already funded, or do we send RUSD first?

Architecture

1 TASK	agent is given a job	
1b QUOTE	(cut first if short on time) buyer gets 2-3 competing quotes,	
		one counter-offer round, winning quote becomes the PO
	v	
2 PROPOSE	agent declares the accepted PO	
		{ poNumber, vendor, sku, unitPrice, quantity, quoteExpiry }
	v	
3 VERIFY	deterministic checks against the record	<-- no model here
		rules are versioned CONFIG, not code
	pass fail	
		\

		\--> REFUSE no card. plain-English reason. logged.
	v	
4 ISSUE		Rain issues a scoped virtual card
		bound to exactly this P0
	v	
5 SETTLE		the purchase happens on that card
	v	
6 RECORD		P0 + checks + rule version + card + outcome, stored, append-only
7 REVOKE		(if the endpoint exists) card deactivated once the job is done

Monad: hash each **rule version** and anchor it on testnet, one transaction per version. This is structural, it is what proves the rules were not edited after the fact to fit history. A per-decision anchor is optional on top, cut first if short on time.

Who owns what

	A	B
Owns	Rain auth, card issuance, collateral draw, the spending agent, the quote stage	The six checks, storage, fixture data, rule versioning + replay, UI, diagram
Blocked by Rain?	Yes	No, start immediately

Join at 17:00. A real check result decides whether a real card gets issued. Not joined by 18:00, cut scope, do not push the join later.

The six checks

1. PO exists and is accepted
2. PO still open, not already fulfilled
3. Amount matches the quote
4. Merchant matches the quote (*secondary point, see below*)
5. Within remaining budget
6. **No card already issued for this PO** (*idempotency, this is the one that carries the demo*)

Each check returns pass/fail plus a plain-English reason, never “validation failed.”

⚠ On check 4: only claim “Rain would have let this through” if the engineer confirms category-level locking. If Rain locks to the exact merchant, say instead “we check the same thing Rain does, plus the parts of the deal Rain never sees.”

The demo, four minutes, in order

1. **Run the task.** Card issued, scoped to the PO, real money moves.
 2. **Press the same button again.** No new fixture, no second agent. **Refused, by check 6, reading the record the first run itself wrote.** This is the headline moment, it cannot be called staged, because nothing about it was scripted for the demo.
 3. **Open the provenance panel on that refusal.** Four fields: the rule that failed, value expected, value got, the record it read. Five seconds, no questions needed.
 4. **Open the rule editor. Edit a rule. New version. Hit replay.** *“Across our history, N approvals would now be refused, M refusals would now pass.”* This needs **seeded history, 40-50 decisions**, or it reads as a toy with six rows.
 5. **Stretch, if built:** hand the judge the keyboard, let them edit a PO field and press issue themselves.
 6. **Close:** *“Rain enforces this at issuance. We enforce the reason at the same point. And because it’s deterministic, we can prove the rules weren’t changed to fit the history.”*
-

Cut order, in this exact sequence

1. **Negotiation/quote stage first.** One seller quote instead of competing ones. This comes before Monad now, not after, it is the most fakeable thing on screen and it sits in front of the judge whose job is agent orchestration.
 2. **Per-decision Monad anchor second.** The rule-version anchor stays if at all possible, it is structural.
 3. Card revoke, budget meter, judge-editable form, in that order.
 4. **Never cut:** issuance + refusal working end to end, the run-it-twice moment, seeded replay, provenance panel, storage that survives deploy, submission itself.
-

Schedule

Time	A	B
now-14:00	Ask the Rain engineer questions above, together	Pick storage, wire it
13:00-15:00	One successful authenticated Rain call. Nothing else matters yet.	Six checks + tests against fixtures, no UI
15:00-17:00	Issue a card with a limit, end to end	UI, provenance panel, seed 40-50 historical decisions
17:00	 JOIN. Real check result decides real issuance.	
17:00-18:00	Both on the join, cut scope if not working by 18:00	

Time	A	B
18:00-20:00	Rule-version Monad anchor, run-it-twice demo path, README	
20:00-21:00	Deploy. Verify replay works on the deployed URL from a phone. Rehearse twice, timed.	

Sunday: fix what broke overnight, nothing new, until 11:00. Submit as soon as it's genuinely ready, pitching order follows submission order, so early beats buffered.

Repo

`github.com/theomthakur/raingentic-team10`, public, `docs/` has the full reasoning behind every line above. `.env.local` never gets committed, only `.env.local.example`.